

本节讲解的例题及其囊括的知识点，如表3-2所示。

表3-2 组合计数例题归纳

编 号	题 号	标 题	知 识 点	代 码 作 者
例 3-16	UVa 1262	Password	编码解码问题	刘汝佳
例 3-17	UVa 10213	How Many Pieces of Land	支持四则运算的大整数类；平面的欧拉定理；计数	刘汝佳
例 3-18	UVa 1645	Count	有根树计数	陈锋
例 3-19	UVa 1649	Binomial coefficients	反向计算二项式系数	陈锋
例 3-20	UVa 11806	Cheerleaders	容斥原理	陈锋
例 3-21	UVa 1069	Always an Integer	多项式；差分；整除性	刘汝佳

3.3 概率与期望

很多和概率相关的问题并不需要特别的知识，熟悉排列组合就够了。

例3-22 【离散条件概率】条件概率（Probability Given, UVa 11181）

有 n 个人准备去逛超市，其中第 i 个人买东西的概率是 P_i 。逛完以后你得知有 r 个人买了东西，根据这一信息，请计算每个人实际买东西的概率。输入 n （ $1 \leq n \leq 20$ ）和 r （ $0 \leq r \leq n$ ），输出每个人实际买东西的概率。

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>

using namespace std;
#define _for(i, a, b) for (int i = (a); i < (b); ++i)
typedef long long LL;
const int MAXN = 20 + 4;
int N, R;
double P[MAXN], P1[MAXN], B;
/*
     $P(A|B) = P(AB) / P(B)$ 
    for i, A|B : i buy; AB: 买 r 个且包含第 i 个; B : 买 r 个, P1=P(AB)
*/
int main() {
    for (int t = 1; scanf("%d%d", &N, &R) == 2 && N; t++) {
        printf("Case %d:\n", t);
        B = 0;
        _for(i, 0, N) P1[i] = 0, scanf("%lf", &(P[i]));
        _for(s, 0, (1 << N)) {
            int r = 0, sc = s;
            while (sc) r++, sc = sc & (sc - 1); // s 中元素个数
            if (R != r) continue; // 元素个数不对
            double p = 1;
            _for(i, 0, N) p *= (s & (1 << i)) ? P[i] : 1 - P[i];
            B += p;
            _for(i, 0, N) if (s & (1 << i)) P1[i] += p;
        }
        _for(i, 0, N) printf("%.6lf\n", P1[i] / B);
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典(第2版)》例题10-11
同类题目: Catch the Plane, ACM/ICPC WF 2018, Codeforces Gym 102482A
          Headshot, ACM/ICPC NEERC 2009, UVa 1636
*/

```

例3-23 【离散概率；DP】纸牌游戏 (Double Patience, NEERC 2005, UVa 1637)

36张牌分成9堆，每堆4张牌。每次可以拿走某两堆顶部的牌，但需要点数相同。如果有多种拿法，则等概率地随机拿。例如，9堆牌的顶部分别为KS, KH, KD, 9H, 8S, 8D, 7C, 7D, 6H，则有5种拿法，分别为(KS, KH), (KS, KD), (KH, KD), (8S, 8D), (7C, 7D)，每种拿

法的概率均为 $1/5$ 。如果最后拿完所有牌，则游戏成功。按顺序给出每堆牌的4张牌，求游戏成功概率。

【代码实现】

```

// 陈锋
#include <cstdio>
#include <algorithm>
using namespace std;
typedef long long LL;

struct Stat {
    int C[9];
    Stat() { fill_n(C, 9, 4); }
    int index() const { // 状态压缩编码
        int ans = 0;
        for (int i = 0; i < 9; i++) ans = ans * 5 + C[i];
        return ans;
    }
};

const int MAXS = 1953125 + 4; // 5^9
double Ans[MAXS];
char Piles[9][4][3];

double DP(Stat& st) {
    double& ans = Ans[st.index()];
    if (ans != -1.0) return ans;
    ans = 0;
    int cnt = 0, empty = 1;
    for (int i = 0; i < 9; i++) {
        int& ci = st.C[i];
        if (ci < 1) continue;
        empty = 0;
        for (int j = i + 1; j < 9; j++) {
            int& cj = st.C[j];
            if (cj < 1) continue;
            if (Piles[i][ci - 1][0] == Piles[j][cj - 1][0]) {
                ci -= 1, cj -= 1; // 后续的一种可能局面
                ans += DP(st);
                cnt++;
                ci += 1, cj += 1;
            }
        }
    }

    if (empty) return ans = 1; // 都已经空了
    if (cnt == 0) return ans = 0; // 必输
    return ans /= cnt;
}

int main() {
    while (true) {
        for (int i = 0; i < 9; i++)
            for (int j = 0; j < 4; j++)
                if (scanf("%s", Piles[i][j]) != 1) return 0;
        fill_n(Ans, MAXS, -1.0);
        Stat st;
        printf("%.6lf\n", DP(st));
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题10-12
同类题目：Collecting Bugs, POJ 2096
*/

```

例3-24 【离散概率；递推】麻球繁衍 (Tribbles, UVa 11021)

有 k 只麻球，每只活一天就会死亡，临死之前可能会生出一些新的麻球。具体来说，生 i 个麻球的概率为 P_i 。给定 m ，求 m 天后所有麻球均死亡的概率（ $1 \leq n \leq 1000$ ， $0 \leq k \leq 1000$ ， $0 \leq m \leq 1000$ ）。注意，不足 m 天时已全部死亡的情况也计算在内。

【分析】

由于每只麻球的后代可独立存活，所以只需求出一开始只有1只麻球， m 天后全部死亡的概率 $f(m)$ 。由全概率公式可知：

$$f(i) = P_0 + P_1 f(i-1) + P_2 f(i-1)^2 + P_3 f(i-1)^3 + \dots + P_{n-1} f(i-1)^{n-1}$$

所以答案是 $f(m)^k$ 。

【代码实现】

```

// 陈锋
#include <cmath>
#include <stdio>
using namespace std;
typedef long long LL;
const int MAXN = 1000 + 4;
double P[MAXN], F[MAXN];
int main() {
    int T;
    scanf("%d", &T);
    for (int t = 1, n, k, m; t <= T; t++) {
        scanf("%d%d%d", &n, &k, &m);
        for (int i = 0; i < n; i++) scanf("%lf", &(P[i]));
        F[0] = 0, F[1] = P[0];
        for (int x = 2; x <= m; x++) {
            F[x] = 0;
            for (int i = 0; i < n; i++) F[x] += P[i] * pow(F[x - 1], i);
        }
        printf("Case #d: %.7lf\n", t, pow(F[m], k));
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.5 节例题 18
同类题目：Scout YYF I, POJ 3744
*/

```

例3-25 【数学期望；有理数】优惠券（Coupons, UVa 10288）

某种彩票一元钱一张，购买后撕开上面的锡箔，你会看到一个漂亮的图案。图案有 n 种，如果你收集全这 n ($n \leq 33$) 种图案，就可以得大奖。请问，在平均情况下，需要买多少张彩票才能得到大奖呢？如 $n=5$ 时，答案为 $137/12$ 。

【分析】

假设已有 k 个图案，令 $s=k/n$ ，得到一个新的图案需要 t 次的概率为 $s^{t-1}(1-s)$ ，因此平均需要的次数为 $(1-s)(1+2s+3s^2+4s^3+\dots)=(1-s)E$ ，而 $sE=s+2s^2+3s^3+\dots=E-(1+s+s^2+\dots)$ ，移项可得

$$(1-s)E=1+s+s^2+\dots=1/(1-s)=n/(n-k)$$

换句话说，已有 k 个图案时，平均再拿 $n/(n-k)$ 次就可多搜集一个图案，所以总次数为

$$n(1/n + 1/(n-1) + 1/(n-2) + \cdots + 1/2 + 1/1)$$

【代码实现】

```

// 陈锋
#include <bits/stdc++.h>
using namespace std;
typedef long long LL;

LL gcd(LL a, LL b) { return b == 0 ? a : gcd(b, a % b); }
struct Rational {          // 有理数类
    LL a, b;                // a/b
    Rational& operator+=(const Rational& r) {
        LL na = a * r.b + b * r.a, nb = b * r.b;
        a = na, b = nb;
        reduce();
        return *this;
    }

    void reduce() {
        LL g = gcd(a, b);
        a /= g, b /= g;
    }
};

int lenOf(LL x) {
    static char buf[32];
    sprintf(buf, "%lld", x);
    return strlen(buf);
}

ostream& operator<<(ostream& os, const Rational& r) {
    if (r.b == 1) return os << r.a;
    LL a = r.a % r.b, b = r.b, k = r.a / r.b;
    int la = lenOf(a), lb = lenOf(b), lk = lenOf(k);
    lk = k == 0 ? 0 : lk + 1, la = max(la, lb);
    if (lk)
        for (int i = 0; i < lk; ++i) os << " ";
    os << a << endl;
    if (lk) os << k << " ";

    for (int i = 0; i < la; ++i) os << "-";
    os << endl;
    if (lk)
        for (int i = 0; i < lk; ++i) os << " ";
    return os << b;
}

int main() {
    for (int n; scanf("%d", &n) == 1; ) {
        Rational r = {n, n};
        for (int i = 1; i < n; i++) r += (Rational) {n, i};
        cout << r << endl;
    }
}
/*
算法分析请参考：《算法竞赛入门经典（第2版）》例题10-18
注意：如何重载Rational的iostream << 操作
同类题目：Pock, HDU 5984
*/

```


例 3-26 【数学期望】玩纸牌 (Expect the Expected, UVa 11427)

每天晚上你都玩纸牌游戏，如果第一次就赢了便高高兴兴地去睡觉，如果输了就继续玩。假设每盘游戏你获胜的概率都是 p （ p 的分母不超过1000），且各盘游戏的输赢是独立的。你是一个固执的完美主义者，因此会一直玩到当晚获胜局数的比例严格大于 p 时才停止，然后高高兴兴地去睡觉。当然，晚上的时间有限，最多只能玩 n （ $1 \leq n \leq 100$ ）盘游戏，如果获胜比例一直不超过 p 的话，你只能垂头丧气地去睡觉，以后再也不玩纸牌了。你的任务是计算出平均情况下你会玩多少个晚上的纸牌。

【代码实现】

```

// 刘汝佳
#include <cmath>
#include <cstdio>
#include <cstring>
const int NN = 100 + 5;
double D[NN][NN];
int main() {
    int T;
    scanf("%d", &T);
    for (int kase = 1, n, a, b; kase <= T; kase++) {
        scanf("%d/%d%d", &a, &b, &n); // 请注意 scanf 的技巧
        double p = (double)a / b;
        memset(D, 0, sizeof(D));
        D[0][0] = 1.0, D[0][1] = 0.0;
        for (int i = 1; i <= n; i++)
            for (int j = 0; j * b <= a * i; j++) {
                // 等价于枚举满足 j/i <= a/b 的 j, 同时避免了除法误差
                double &d = D[i][j];
                d = D[i - 1][j] * (1 - p);
                if (j) d += D[i - 1][j - 1] * p;
            }
        double Q = 0.0;
        for (int j = 0; j * b <= a * n; j++) Q += D[n][j];
        printf("Case #%d: %d\n", kase, (int)(1 / Q));
    }
    return 0;
}
/*
算法分析请参考:《算法竞赛入门经典——训练指南》升级版 2.5 节例题 20
同类题目: POJ3682 King Arthur's Birthday Celebration
*/

```

例3-27 【马尔可夫过程；数学期望】得到1 (Race to 1, UVa 11762)

给出一个整数 N ($1 \leq N \leq 1\,000\,000$)，每次可以在不超过 N 的素数中随机选择一个 P ，如果 P 是 N 的约数，则把 N 变成 N/P ，否则 N 不变。计算平均情况下需要多少次随机选择，才能把 N 变成1。例如， $N=3$ 时，答案为2； $N=13$ 时，答案为6。

【代码实现】

```

// 刘汝佳
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <cstring>
using namespace std;

const int NN = 1e6 + 10;
double F[NN];
int IsPrime[NN], primes[NN], vis[NN];

void gen_primes(int n) { // 素数筛法
    fill_n(IsPrime, n + 1, 1);
    for (int i = 2, p = 0; i <= n; i++) {
        if (!IsPrime[i]) continue;
        primes[p++] = i;
        if (i <= n / i)
            for (int j = i * i; j <= n; j += i) IsPrime[j] = 0;
    }
}

double dp(int x) {
    double& f = F[x];
    if (x == 1) return 0.0; // 边界
    if (vis[x]) return f; // 记忆化
    vis[x] = 1;
    int g = 0, p = 0; // 累加 g(x) 和 p(x)
    f = 0;
    for (int i = 0; primes[i] <= x; i++) {
        p++;
        if (x % primes[i] == 0) g++, f += dp(x / primes[i]);
    }
    return f = (f + p) / g;
}

int main() {
    int T;
    scanf("%d", &T);
    gen_primes(NN - 1), fill_n(vis, NN, 0);
    for (int kase = 1, n; kase <= T; kase++) {
        scanf("%d", &n);
        printf("Case %d: %.10lf\n", kase, dp(n));
    }
    return 0;
}
/*
算法分析请参考：《算法竞赛入门经典——训练指南》升级版 2.5 节例题 21
同类题目：Wandering Robots, HDU 6229
*/

```